

Efficient Software Development with UML-Based Code Generation

Dipmala Salunke

Information Technology

Rajarshi Shahu College of Engineering
Pune, India

dtsalunke_it@jspmrscoe.edu.in

Om Shinde

Information Technology

Rajarshi Shahu College of Engineering
Pune, India

shindeom1513@gmail.com

Akash Nipunge

Information Technology

Rajarshi Shahu College of Engineering
Pune, India

nipungeakash9359@gmail.com

Darshan Patil

Information Technology

Rajarshi Shahu College of Engineering
Pune, India

darshanpatil972@gmail.com

Pallavi Tekade

Information Technology

Rajarshi Shahu College of Engineering
Pune, India

pmtekade_it@jspmrscoe.edu.in

Swapnil Nikat

Information Technology

Rajarshi Shahu College of Engineering
Pune, India

swapnilnikat9399@gmail.com

Abstract—This study examines the shortcomings and restrictions of web-based platforms for developing applications, including website builders, no-code app builders, and e-commerce platforms, in terms of their ability to satisfy the distinct needs and rationale of various business types. Even if these platforms are practical and simple to use, they frequently fall short in offering tailored software solutions that meet the unique requirements of companies. The research emphasizes how difficult it is for startups and small companies to find qualified engineers and how expensive it is to keep up internal development teams. The dangers and restrictions of using website builders and comparable platforms are also covered, including the possibility of vendor lock-in, problems with data control, and a lack of flexibility when it comes to adding special features or procedures. The goal of this research is to offer useful insights for companies who want to use web-based applications in their operations by identifying and analyzing these limits. In the end, it aims to further knowledge of the constraints of current development platforms and assist companies in making decisions that will maximize their online presence and operations.

Index Terms—Unified Modeling Language (UML), Software Development Life Cycle (SDLC), Machine Learning (ML), Natural Language Processing (NLP), Model-Driven Development, Requirements Engineering, Object-Oriented Design, Code Generation, Development Efficiency, Prototyping, Use Case Diagrams.

I. INTRODUCTION

In the current digital era, web-based applications are critical to the growth and success of businesses and startups in every field. These applications are the cornerstone of every business operation, helping companies connect and communicate with their target markets, improve workflows, and inspire innovation.

Hiring staff for web application development is an exhausting task that may not be possible for smaller businesses and start-ups for many reasons. First, recruiting skilled developers, designers, and other professionals may be quite expensive, especially for businesses with limited resources. Also, ongoing

expenses related to overhead, perks, and salaries can mount up quickly, straining small firms' limited funding sources.

Without writing a single line of code, platforms like Wix, Squarespace, and Weebly allow small businesses to create drag-and-drop websites with a professional appearance. These systems include hosting services for hassle-free deployment, editable templates, integrated e-commerce, blogging, and SEO functions. Businesses that use website builders face the risk of becoming dependent on the platform's unique tools and hosting services, which can make it difficult to relocate or expand their online presence in the future. Rebuilding the website from the very beginning up may be necessary when switching to a new platform, which could cause downtime and data loss. Businesses using these tools may have limited control over their data, including customer information, sales data, and product listings. Platform policies may dictate how data is stored, accessed, and utilized, potentially restricting businesses' ability to leverage data for marketing or analytics purposes.

One significant disadvantage of website builders, no-code app builders, and e-commerce platforms is their inability to provide software solutions that make sense for a business's particular logic. These platforms typically come with pre-built modules, themes, and features that might not be fully tailored to the unique requirements and complexity of various business kinds. Consequently, businesses may find that the pre-existing frameworks of the platforms impede their ability to incorporate distinctive features or processes that are essential to their daily operations.

By exploring the drawbacks of these platforms in greater detail—especially with regard to their inability to offer customized software solutions that satisfy the distinct needs and logic of different businesses—this paper seeks to advance the body of information already in existence. Through the identification and analysis of these constraints, this study aims to provide valuable information that can assist companies

looking to use web-based apps in their operations to make informed decisions.

II. ARTIFICIAL INTELLIGENCE

The goal of artificial intelligence (AI), a branch of computer science, is to create intelligent machines that can do activities that typically require human intelligence. Learning, thinking, problem-solving, perception, comprehending spoken language, and even interacting with the surroundings are some of these skills.[2] Artificial intelligence (AI) systems are made to mimic cognitive processes like problem-solving and decision-making; they frequently rely on data and algorithms to provide predictions. Over time, these systems can be trained to identify patterns in huge datasets, which will improve their accuracy in performing tasks. Artificial intelligence (AI) can be approached from different angles. Symbolic AI, for example, requires programming machines to obey logical principles and draw conclusions. Machine learning, on the other hand, allows systems to learn from data without explicit programming.

III. OVERVIEW OF NLP

The study of human language processing and analysis via computing methods is known as natural language processing, or NLP. NLP is the umbrella term for a variety of methods and strategies used to interpret, represent, and work with text in natural language. These strategies cover a spectrum from early grammar-based approaches like pattern matching and Context-Free Grammar (CFG) to more sophisticated statistical approaches, dimensionality reduction strategies, and neural network-based word embeddings. Natural language processing (NLP) has undergone substantial evolution to handle the complexity of natural language interpretation and processing. It is crucial for tasks like information retrieval, chatbots, machine translation, and language production[3].

NLP Working –

- **Preprocessing:** Raw language input is handled and made ready for the NLP model to use at the first stage of the NLP pipeline. To provide uniformity and standardization, this entails cleaning, normalization, and tokenization of the text data.
- **Mapping:** The text is mapped into an appropriate format that the NLP model can comprehend and interpret following preprocessing. In this step, the text data is transformed into an inputable structured representation for the model, like numerical vectors or matrices.
- **Embedding:** The process of embedding involves transforming the original mapped information into a format that the model can use. Typically, this entails methods like word embeddings, which capture the semantic links between words by representing words as dense vectors in a continuous vector space.
- **Model Training:** After preprocessing, mapping, and embedding the data, it is input into the NLP model. In the training phase, the model picks up characteristics and parameters from the input data that it will need for tasks like language synthesis later on.

- **Language Generation:** Following training, the model can be applied to problems involving the creation of natural language. Using methods like sequence-to-sequence models, recurrent neural networks (RNNs), or transformer models like BERT or GPT, this entails producing text that is human-like based on the parameters and attributes that have been learned [5].

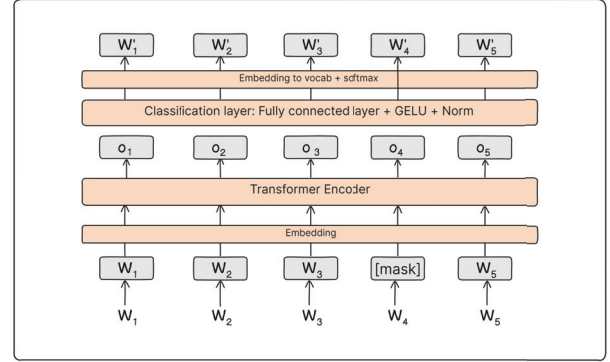


Fig. 1. Working of NLP.

The NLP pipeline, in its entirety, entails preprocessing unprocessed language input, formatting it appropriately, embedding it into a numerical representation, training an NLP model to acquire parameters, and applying the trained model to tasks like language production.[7]

IV. OVERVIEW OF LLM

A class of neural language models (NLMs) known as Large Language Models (LLMs) use massive amounts of data with deep learning methods to make notable progress in natural language creation and processing tasks. Built on top of previous pre-trained neural language models, LLMs are known for their enormous scale—billions of parameters are frequently present. Self-supervised learning techniques are used to train them on large amounts of unlabeled text data.

LLM Working – Data Preprocessing: This step entails getting the input text ready for the LLM to read. There are multiple sub-steps in it:

- **Data cleaning** is the process of eliminating extraneous characters, formatting errors, and other noise from the text input.
- **Noise reduction** is the process of removing any extraneous or distracting content from the text.
- **Data integration** is the process of combining data from several sources, if needed, to produce a single input.

Tokenization: The process of dividing the input text into smaller components, known as tokens, which are usually words or subwords, is known as tokenization.

- Every token has an embedded token vector representation and is given a unique identification (Token Id).
- The token is represented numerically in the embedded token vector, which is an input for the model's later phases.

Pre-Training: During this phase, the LLM gains an understanding of the connections and patterns found in the input text. Several layers of the LLM process the token vectors created at the tokenization stage, including: Layer normalization is a method that normalizes the inputs to each layer in order to stabilize the learning process. Attention Mechanism: Enables the model to process each token by focusing on distinct sections of the input text. The activation function gives the model non-linearity, which helps it understand intricate correlations in the data. The model learns to extract features and comprehend the context of the input text by going through several hidden layers with the processed token vectors.

Ultimately, the model's internal parameters are adjusted throughout the training phase in order to reduce the discrepancy between the expected and actual outputs. This allows for precise parameter tuning.[8]

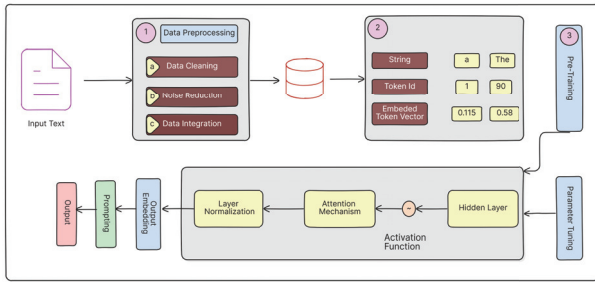


Fig. 2. Working of LLM.

graphic highlights the essential elements and procedures in each step of the process, giving a high-level overview of the steps involved in operating an LLM, from data preprocessing to pre-training.

V. LITERATURE SURVEY

F. Ciccozzi et al. [7] The survey paper explores UML-based development techniques designed for embedded real-time multicore applications. It looks at a number of approaches, including modeling, optimizing job distribution, creating code automatically, and translating ALF to C code for multithreaded applications. Important results from the SMART Core project, a well-known project in this field, are highlighted. The research highlights how crucial it is to use a variety of optimization strategies in order to improve UML-based software development efficiency. It demonstrates how model-based and execution-based task allocation optimization techniques work together to achieve the best possible resource usage across a number of cores. The project also demonstrates the advantages of using UML models to generate code automatically, which speeds up development. The survey highlights the crucial role that real-time performance plays in embedded systems development, with particular emphasis on meeting deadlines and ensuring end-to-end reaction times.

M. Jurgelaitis et al. [8] Concentrates on using Model-Driven Architecture (MDA) concepts and UML diagrams to create Solidity smart contracts from UML models. It describes the

algorithms used to convert models into Ethereum blockchain smart contracts that may be executed. In addition to bespoke algorithms for model transformations and smart contract code generation, the study makes use of Eclipse ATL for Model-to-Model (M2M) transformations and Eclipse Acceleo for Model-to-Text transformations. Proposed methods for model transformations and smart contract code generation are presented in the study along with examples of their successful implementation on a sandbox blockchain. It also provides a comparison analysis between the code that is generated and the original smart contracts. In conclusion, the study provides a thorough examination of using UML diagrams and MDA concepts for Solidity smart contract development, describing techniques and providing real-world code generation and deployment results.

R. Tesoriero et al. [9] The research study presents a model transformation architecture intended to produce source code for full stack web applications. It uses M2T (Model-to-Text) and M2M (Model-to-Model) transformations in accordance with OMG standards and represents software using UML class models. Eclipse plugins are used to combine the architecture with external tools. Using the concepts of Model Driven Architecture (MDA) to create three-layered online applications, the resulting architecture suggests a model transformation strategy for creating multi-layered web applications. The architecture outlines a web service layer that adheres to REST service architecture, a persistence layer that uses the Propel framework for MySQL databases, and a client-server configuration with a PHP server and HTML client. Nevertheless, the architecture has drawbacks, like the persistence layer's support for only MySQL and the REST services with JSON in the web service layer, and Polymer and JavaScript for the presentation layer.

U. K. Acharjee, M. Arefin et al. [11] This paper presents a revolutionary deep learning approach for transforming pseudocode into executable code. The technique makes use of sequence-to-sequence deep learning model trained with a customized dataset called SPoC, as well as text preprocessing based on natural language processing. To improve performance, the method also applies sequence learning algorithms, dataset rearrangement, and input preprocessing. The suggested method works better than current methods in terms of efficiency and accuracy, according to the results. Interestingly, to increase accuracy even further, the method makes use of two Long Short-Term Memory (LSTM) networks. The study does, however, note limits around ambiguity in pseudocode, which may result in the generation of ambiguous code, and difficulties in declaring variable types, which may lead to the creation of inaccurate pseudocode.

Z. Zhu, Y. Lei, Y. Zhu and H. Sarjoughian. [13] This work introduces a novel method for capturing cognitive activities in combat simulation systems: UML profile-based metamodeling. It describes the creation of a domain-specific language (DSL) designed to efficiently represent complex domain knowledge. The research attempts to close the communication gap between domain experts and modelers by constructing an Abstract Syntax Tree (AST) profile. Important techniques include cre-

TABLE I
LITERATURE SURVEY

Sr no.	Year & Reference no	Method	Limitations	Future Scope
1	IEEE-2023 [12]	Static and dynamic code analysis are combined with black-box, white-box, and grey-box testing techniques to thoroughly assess software.	Advanced Java programming abilities are necessary for white-box testing because mistakes or missing methods can cause the testing to stop.	The system focuses on optimizing testing content development and has a layered structure for input, processing, and evaluation. Extension plans are in place for Python and C.
2	IEEE-2022 [18]	UML class diagram for structural reference. State machine that converts XML into code. Maintainability and efficiency of produced code. Evaluating consistency of UML models.	Structured only for class diagrams.	To improve maintainability and efficiency in the process of generating code. Evaluating consistency of UML models and support for diagrams other than class diagrams.
3	IEEE-2022 [8]	Model-to-model and model-to-text translation tools facilitate speedier code generation and improve the development of Solidity smart contracts on Ethereum	Absence of standardized decentralized business process solutions. Because smart contracts are immutable, their development is subject to human error.	Extend the PSM level of approach to include other technologies. Create an expandable collection of pre-defined Solidity code samples.
4	IEEE-2022 [11]	NLP is used for data preparation using a sequence-to-sequence deep learning model from the SPoC dataset, which focuses on text pre-treatment and reorganization.	Uncertainty in pseudocode results in unclear code creation. Incorrect pseudocode can arise from declaring variable types with difficulty.	To look into more sophisticated neural network designs. Examine how transformers might be integrated for improved pseudo-code translation.
5	IEEE-2022 [9]	ACCELEO and ATL power an architecture that complies with OMG standards, integrates with the Eclipse IDE, and generates source code for full-stack WebApps via model transformations.	Persistence layer is limited to MySQL support. Only ReST services with JSON are supported by the web service layer. Only JavaScript and Polymer are supported by the Presentation Layer.	Upcoming research involve refining model transformations, automating UI improvements, filling in research gaps, and incorporating industry standards to improve tool interoperability
6	IEEE-2020 [15]	System engineering using models. Diagrams in SysML and UML. For static ABV, NLCTL. SVOCL for dynamic limitations on verification.	Current frameworks concentrate on either static or dynamic verification independently.	Increase SoS integration by adding the most recent verification standards, such as Stimulus Standard and Portable Test, and supporting automated modeling and test benches.
7	IEEE-2019 [16]	Src2MoF uses IMD to parse Java source code into intermediate models, which are further converted into high-level UML models by a generator engine.	Src2MoF is only supported for OOP languages like Java, etc. Uses UML activity diagram to represent legacy system behavior, etc. but fails to merge and fork node representation.	Improve Src2MoF by adding support for C and making use of more UML diagrams to provide a clearer, more in-depth examination of legacy systems.
8	TÜBİTAK-2020 [19]	Table design and creation. Saving the program design. HTML template selection and plugin creation. JSON generation.	The code generator's lack of flexibility in handling different types of users can be a limitation.	Integration of the new design into TBYS further optimization of the generator code.
9	IEEE-2019 [14]	A method in which predefined patterns are applied to the structure and behavior of a state diagram to create an execution process.	Complexity, limitations, competition for change, quality of problems, tool dependencies, maintenance burden, and learning curve.	Standards development and supportive tools, solving scalability, customization challenges to enable widespread use in software engineering and improving code quality and usability.
10	IEEE-2017 [13]	Metamodeling based on UML profiles. Defining DSLs according to the UML profile. OODA loop approach for defining uncertainty and ambiguity.	Among the drawbacks are the integration of models with disparate syntax and semantics.	The development of languages specific to a certain topic may require extensive metamodeling. Expanding profiles for other domains of cognitive behavior as part of the future scope. Possibility of modifying the framework for creating high-caliber domain profiles.
11	IET-2016 [20]	This process includes DFS traversal of the graph, subroutine calls generated by the defined method, and execution logic defined by the program code.	Narrowness of diagrams and system diagrams. Complex controls can be difficult to manage, and there is no evaluation metrics for effectiveness of generated code. Limited language support is also a limitation.	To cover a wider range of UML diagrams, and improve code generation success. Better handling of complex controls, support for various programming languages and integration of robust verification techniques.
12	IJESRT-2014 [21]	The approach includes pre-configuration, analysis, optimization and code generation, as well as simplification and readability of the generated code.	Only focusing on simplifying, optimizing the UML diagram generated by the programming process may ignore complexity of the real situation and business logic. Low compatibility of system for production ready.	To include additional support for multiple programming languages, improved handling of complex logic, and integration optimization techniques for better code generation.

ating a DSL that is appropriate for full domain knowledge representation, separating cognitive models from simulation models, and optimizing existing languages for DSL creation. The suggested AST profile effectively lowers communication obstacles, according to the results, and the development of cognitive domain-specific profiles is facilitated by a broad framework. However, there are also disadvantages, including the possible necessity for complicated metamodeling during DSL building, the intricacies resulting from competing demands of domain experts and technical modelers, and the challenges of integrating models with disparate syntactic and semantic frameworks.

S. E. V. and P. Samuel. [14] This paper presents a new design pattern for state machine implementation with hierarchy and concurrency that is intended to make it easier to generate code automatically from UML diagrams. In order to compare the performance of the suggested design pattern with well-known programs like Rhapsody and OCode, it is assessed in conjunction with a code generator known as SMConverter. The suggested method produces less difficult code and offers promising results, according to the results. Additionally, the design pattern improves code quality and supports history and state parallelism, which increases event processing efficiency. The report does, however, acknowledge certain shortcomings. To begin with, state diagram elements are frequently not sufficiently supported by current programming languages. Second, results from this field of study frequently do not provide efficient means of executing concurrent states. To make matters more difficult, a lot of the previously published publications fall short of offering a complete design pattern for application. And finally, the literature that is now available does not sufficiently support or effectively portray concurrency and history states. For state machine implementation with hierarchy and concurrency, in particular, the paper offers insightful information about automating code generation from UML despite these drawbacks.

U. Sabir, F. Azam et al. [16] In this paper, the MDRE (Model-Driven Reverse Engineering) framework Src2MoF is presented. Its purpose is to create UML (Unified Modeling Language) models from Java source code. The primary goal of Src2MoF is to capture the structural and behavioral features of legacy systems in order to improve comprehension and streamline maintenance procedures. An analysis of the literature highlights the importance of MDRE in software development and highlights how it may be used to assess legacy systems with UML diagrams. The study's methodologies involve the creation of a Java UML model generator and the source code parsing for Intermediate Model Discoverer (IMD). The findings demonstrate that Src2MoF can effectively produce UML structural and behavioral diagrams from Java code, establishing it as the first tool to model behavior and structure straight from text. The framework's usefulness is intended to be expanded by extending its capabilities to other programming languages, such as C. The study does point out certain shortcomings, though, most notably the lack of adequate tool support for simulating the structural elements of older systems.

Notwithstanding this limitation, the MDRE solution offered by Src2MoF shows its effectiveness in producing high-level models from Java source code, providing insightful information for software engineering practices—particularly when it comes to the analysis and maintenance of older systems.

Y. Park, A. Park and C. Kim. [17] The paper introduces ALSI-Transformer, a unique method that uses both lexical and syntactic alignment to improve code comment generation. With state-of-the-art performance in comment creation tasks, this proposed model outperforms baselines in multiple code summarization criteria. The key component of the technique is the introduction of a new data type called CAT, which makes it easier to align lexical and syntactic data, increasing the precision of comment production. The main model of ALSI-Transformer is based on transformer architecture and it generates code comments using template-based, retrieval-based, and learning-based approaches. The outcomes show that ALSI-Transformer synchronizes lexical and syntactic information well, producing better results in tasks involving the creation of comments. It also exhibits mastery in managing lengthy source code and resolving user-defined identifiers and Out-of-Vocabulary (OOV) words. Nevertheless, the study points out a number of drawbacks, such as possible mistakes in experimental code, difficulties handling complicated source code, and a decreased capacity for comment creation when working with sophisticated code structures. ALSI-Transformer is a noteworthy development in code comment generation, demonstrating how lexical and syntactic alignment can enhance accuracy and performance. Notwithstanding its constraints, the suggested methodology exhibits potential in mitigating the difficulties linked to automated code summarizing and augmenting developer efficiency in software engineering assignments.

VI. FUTURE WORK

In this paper, we have outlined the development of a code generation system from UML diagrams. To further enhance and extend the capabilities of this system, we propose the following future work:

- 1) **Support for Additional Programming Languages:** Extend the system to generate code for more programming languages, including emerging or domain-specific languages, to increase its versatility and adoption in different software development environments.
- 2) **Support for Real-time Collaboration:** Enable real-time collaboration features where multiple developers can work on the same UML diagrams and code generation process simultaneously, facilitating team-based development.
- 3) **AI-assisted Model Refinement:** Integrate AI or machine learning techniques to assist in refining UML diagrams based on historical data or best practices, helping developers create more accurate and efficient models.
- 4) **Comprehensive Testing Support:** Integrate automated testing capabilities that generate test cases from UML

diagrams and validate the functionality and reliability of the generated code.

- 5) **Scalability and Cloud Integration:** Explore scalability solutions and integration with cloud-based platforms to support large-scale projects and distributed development teams.
- 6) **User Feedback and Adaptation:** Implement mechanisms for collecting user feedback on the generated code's quality and utility, using this feedback to continuously improve the system's algorithms and features.

By addressing these areas, the code generation system could become a more comprehensive and powerful tool for developers, contributing significantly to the field of software engineering automation.

VII. CONCLUSION

The primary goal of this project is to improve developer productivity and optimize software development processes by automating code generation from UML diagrams. This study advances the state-of-the-art in automatic code generation methodologies by emphasizing new techniques like leveraging UML diagrams in image or PDF format and bundling the complete project based on the given framework.

By examining current research and approaches, it has been determined that code generation from UML diagrams automatically offers great potential to accelerate software development cycles, decrease human labor, and enhance code quality. However, compatibility, complexity, and scalability issues plague many of the current methods. Some promising ways to address these issues include the creative approaches suggested in this study, such as using UML diagrams in PDF or picture format and grouping all projects together into a single framework. Developers can expedite the software development lifecycle and promote increased stakeholder participation by employing these strategies to convert visual UML models into executable code.

In conclusion, this study makes a significant contribution to the field of automatic code generation from UML diagrams by offering insightful analysis and useful solutions. We pave the path for more effective and efficient software development techniques by embracing novel ways and addressing significant obstacles. This advances the field and benefits stakeholders and developers equally.

REFERENCES

- [1] J. Estevez, G. Garate and M. Graña, "Gentle Introduction to Artificial Intelligence for High-School Students Using Scratch," in *IEEE Access*, vol. 7, pp. 179027-179036, 2019, doi: 10.1109/ACCESS.2019.2956136.
- [2] R. Patil, S. Boit, V. Gudivada and J. Nandigam, "A Survey of Text Representation and Embedding Techniques in NLP," in *IEEE Access*, vol. 11, pp. 36120-36146, 2023, doi: 10.1109/ACCESS.2023.3266377.
- [3] G. Attigeri, A. Agrawal and S. V. Kolekar, "Advanced NLP Models for Technical University Information Chatbots: Development and Comparative Analysis," in *IEEE Access*, vol. 12, pp. 29633-29647, 2024, doi: 10.1109/ACCESS.2024.3368382.
- [4] M. A. K. Raiaan *et al.*, "A Review on Large Language Models: Architectures, Applications, Taxonomies, Open Issues and Challenges," in *IEEE Access*, vol. 12, pp. 26839-26874, 2024, doi: 10.1109/ACCESS.2024.3365742.
- [5] M. Omar, S. Choi, D. Nyang and D. Mohaisen, "Robust Natural Language Processing: Recent Advances, Challenges, and Future Directions," in *IEEE Access*, vol. 10, pp. 86038-86056, 2022, doi: 10.1109/ACCESS.2022.3197769.
- [6] Large Language Models: A Survey By Shervin Minaee, Tomas Mikolov, Narjes Nikzad, Meysam Chenaghlu Richard Socher, Xavier Amatriain, Jianfeng Gao (<https://arxiv.org/pdf/2402.06196.pdf>)
- [7] F. Ciccozzi, T. Seceleanu, D. Corcoran and D. Scholle, "UML-Based Development of Embedded Real-Time Software on Multi-Core in Practice: Lessons Learned and Future Perspectives," in *IEEE Access*, vol. 4, pp. 6528-6540, 2016, doi: 10.1109/ACCESS.2016.2604018.
- [8] M. Jurgelaitis, L. čepionienė and R. Butkienė, "Solidity Code Generation From UML State Machines in Model-Driven Smart Contract Development," in *IEEE Access*, vol. 10, pp. 33465-33481, 2022, doi: 10.1109/ACCESS.2022.3162227.
- [9] R. Tesoriero, A. Rueda, J. A. Gallud, M. D. Lozano and A. Fernando, "Transformation Architecture for Multi-Layered WebApp Source Code Generation," in *IEEE Access*, vol. 10, pp. 5223-5237, 2022, doi: 10.1109/ACCESS.2022.3141702.
- [10] A. T. Imam, "The Automatic Definition of the Intuitive Linguistic Heuristics Set to Recognize the Elements of UML Analysis and Design Models in English," in *IEEE Access*, vol. 11, pp. 93381-93392, 2023, doi: 10.1109/ACCESS.2023.3310394.
- [11] U. K. Acharjee, M. Arefin, K. M. Hossen, M. N. Uddin, M. A. Uddin and L. Islam, "Sequence-to-Sequence Learning-Based Conversion of Pseudo-Code to Source Code Using Neural Translation Approach," in *IEEE Access*, vol. 10, pp. 26730-26742, 2022, doi: 10.1109/ACCESS.2022.3155558.
- [12] J. Skalka and M. Drlik, "Development of Automatic Source Code Evaluation Tests Using Grey-Box Methods: A Programming Education Case Study," in *IEEE Access*, vol. 11, pp. 106772-106792, 2023, doi: 10.1109/ACCESS.2023.3317694.
- [13] Z. Zhu, Y. Lei, Y. Zhu and H. Sarjoughian, "Cognitive Behaviors Modeling Using UML Profile: Design and Experience," in *IEEE Access*, vol. 5, pp. 21694-21708, 2017, doi: 10.1109/ACCESS.2017.2760060.
- [14] S. E. V. and P. Samuel, "Automatic Code Generation From UML State Chart Diagrams," in *IEEE Access*, vol. 7, pp. 8591-8608, 2019, doi: 10.1109/ACCESS.2018.2890791.
- [15] M. W. Anwar, M. Rashid, F. Azam, A. Naeem, M. Kashif and W. H. Butt, "A Unified Model-Based Framework for the Simplified Execution of Static and Dynamic Assertion-Based Verification," in *IEEE Access*, vol. 8, pp. 104407-104431, 2020, doi: 10.1109/ACCESS.2020.2999544.
- [16] U. Sabir, F. Azam, S. U. Haq, M. W. Anwar, W. H. Butt and A. Amjad, "A Model Driven Reverse Engineering Framework for Generating High Level UML Models From Java Source Code," in *IEEE Access*, vol. 7, pp. 158931-158950, 2019, doi: 10.1109/ACCESS.2019.2950884.
- [17] Y. Park, A. Park and C. Kim, "ALSI-Transformer: Transformer-Based Code Comment Generation With Aligned Lexical and Syntactic Information," in *IEEE Access*, vol. 11, pp. 39037-39047, 2023, doi: 10.1109/ACCESS.2023.3268638.
- [18] D.C.Apostol, P.D.Rusovan, Marius Marcu, "UML to code, and code to UML, a view inside implementation challenges and cost" in *IEEE Access*, vol. 11, pp. 39037-39047, 2023, doi: 10.1109/ICSTCC55426.2022.9931871.
- [19] UYANIK, BURAK and ŞAHİN, VEYSEL HARUN (2020) "A template-based code generator for web applications, Turkish Journal of Electrical Engineering and Computer Sciences: Vol. 28: No. 3, Article 37. <https://doi.org/10.3906/elk-1910-44> Available at: <https://journals.tubitak.gov.tr/elektrik/vol28/iss3/37>
- [20] S. E. V. and P. Samuel, "Automatic code generation using unified modeling language activity and sequence models" in *IET Software*, 2016, vol. 10, ss. 6, pp. 164-172, doi: 10.1049/iet-sen.2015.0138 www.ietdl.org.
- [21] Deepak T. Mane , Rahul Moorthy , Prashant Kumbharkar , Gopal Upadhye,Rashmi Ashtagi," Pattern Classification using Supervised Hypersphere Neural Network", International Journal of Emerging Technology and Advanced Engineering, Volume 12, Issue 08, August 2022
- [22] Harshal Kapase, Kalyani Galande, Tanmay Sonna, Deepali Pawar, A Review On: Sentiment Polarity Analysis On Twitter Data From Different Events, International Research Journal Of Engineering And Technology (IRJET),Volume 5, Issue 3,2018,pp. 1479-1482.
- [23] Harshal Kapase, Dipmala Salunke, Kalyani Galande, Tanmay Sonna, Deepali Pawar, A Review On: Sentiment Polarity Analysis On Twitter Data From Different Events, International Research Journal Of Engineering And Technology (IRJET),Volume 5, Issue 3,2018,pp. 1479-1482.